Continues Assessment - 5

NotPetya Ransomware Attack (2017):

1. **Introduction:** The NotPetya attack was one of the most destructive cyber incidents in history. Although it appeared to be ransomware, it was actually a wiper malware designed to destroy data permanently.

2. Method of Attack:

2.1 Initial Infection

The attack began through a compromised update of M.E.Doc, widely used in Ukraine.

2.2 Exploitation Techniques

- Used EternalBlue
- Credential stealing tools (Mimikatz)
- Admin tools (PsExec, WMIC)

2.3 Payload Behavior

- Overwrites Master Boot Record (MBR)
- Encrypts file system structures
- Makes recovery impossible

3. Spread of the Attack:

- Rapid spread via:
- Network connections
- Shared credentials
- Internal systems
- Affected multiple countries within hours

4. Impact on Organizations:

4.1 Case Study: Maersk:

Effects:

- Shutdown of global IT systems
- 45,000 PCs and 4,000 servers affected
- Port operations stopped worldwide

Financial Loss:

- Approx. $300 million loss

4.2 Other Affected Companies:

- Merck & Co.
- FedEx
- Rosneft

5. Total Damage:

- Estimated global loss: $10 billion

6. Response and Recovery (Maersk):

6.1 Major Challenge

- No clean backups available

- Entire network infrastructure corrupted

6.2 Recovery Strategy

- Rebuilt IT systems from scratch
- Used a surviving server from Ghana office

6.3 Recovery Time

- ~10 days to resume operations
- Full recovery took months

7. Lessons Learned:

- Maintain offline backups
- Improve network segmentation
- Apply regular security patches
- Monitor internal network activity

ElGamal Encryption Algorithm

1. Introduction:

ElGamal is a public-key encryption algorithm based on the Discrete Logarithm Problem, widely used in cryptographic systems.

2. Algorithm Steps:

2.1 Key Generation:

- Choose large prime number p
- Select generator g
- Choose private key x

- Compute public key: $y = g^x \bmod p$
- Public Key: (p, g, y)
- Private Key: x

Encryption:

For message m:
$c1 = g^k \bmod p$
$c2 = m \times y^k \bmod p$

Ciphertext: $(c1, c2)$

Decryption:
$s = c1^x \bmod p$
$m = c2 \times s^{-1} \bmod p$

3. Implementation (Python Code):

```
import random

def mod_inverse(a, p):
    return pow(a, -1, p)

def generate_keys(p, g):
    x = random.randint(1, p-2)
    y = pow(g, x, p)
    return (p, g, y), x

def encrypt(public_key, m):
    p, g, y = public_key
    k = random.randint(1, p-2)
    c1 = pow(g, k, p)
    c2 = (m * pow(y, k, p)) % p
    return (c1, c2)

def decrypt(private_key, public_key, ciphertext):
    p, g, y = public_key
    c1, c2 = ciphertext
    s = pow(c1, private_key, p)
    m = (c2 * mod_inverse(s, p)) % p
```

```
    return m

# Example Execution
p = 23
g = 5

public_key, private_key = generate_keys(p, g)

message = 10
cipher = encrypt(public_key, message)
decrypted = decrypt(private_key, public_key, cipher)

print("Original:", message)
print("Encrypted:", cipher)
print("Decrypted:", decrypted)
```

Output Example:

Original Message: 10
Encrypted: (8, 3)
Decrypted: 10

4. Advantages:

- Strong security
- Randomized encryption.

5. Limitations:

- Larger ciphertext size
- Slower than symmetric encryption.

Implementation of code :

main.py

```python
import random

def mod_inverse(a, p):
    return pow(a, -1, p)

def generate_keys(p, g):
    x = random.randint(1, p-2)
    y = pow(g, x, p)
    return (p, g, y), x

def encrypt(public_key, m):
    p, g, y = public_key
    k = random.randint(1, p-2)
    c1 = pow(g, k, p)
    c2 = (m * pow(y, k, p)) % p
    return (c1, c2)

def decrypt(private_key, public_key, ciphertext):
    p, g, y = public_key
    c1, c2 = ciphertext
    s = pow(c1, private_key, p)
    m = (c2 * mod_inverse(s, p)) % p
    return m

# Example Execution
p = 23
g = 5

public_key, private_key = generate_keys(p, g)

```

Output

```
Original: 10
Encrypted: (8, 5)
Decrypted: 10

=== Code Execution Successful ===
```


```python
    x = random.randint(1, p-2)
    y = pow(g, x, p)
    return (p, g, y), x

def encrypt(public_key, m):
    p, g, y = public_key
    k = random.randint(1, p-2)
    c1 = pow(g, k, p)
    c2 = (m * pow(y, k, p)) % p
    return (c1, c2)

def decrypt(private_key, public_key, ciphertext):
    p, g, y = public_key
    c1, c2 = ciphertext
    s = pow(c1, private_key, p)
    m = (c2 * mod_inverse(s, p)) % p
    return m

# Example Execution
p = 23
g = 5

public_key, private_key = generate_keys(p, g)
```


```python

message = 10
cipher = encrypt(public_key, message)
decrypted = decrypt(private_key, public_key, cipher)

print("Original:", message)
print("Encrypted:", cipher)
print("Decrypted:", decrypted)
```


```
Original: 10
Encrypted: (8, 5)
Decrypted: 10

=== Code Execution Successful ===
```